MAESTRÍA EN

INTELIGENCIA ARTIFICIAL

NOMBRE DE LA MATERIA
APRENDIZAJE AUTOMÁTICO AVANZADO

UNIDAD 1 ACTIVIDAD 2

NOMBRE DEL DOCENTE
DR. IVÁN CASTILLO ZÚÑIGA

NOMBRE DEL ESTUDIANTE
HERNÁN IVÁN MONTOYA MOTATO

TÍTULO DE LA ACTIVIDAD
FUNDAMENTOS DEL APRENDIZAJE
AUTOMÁTICO AVANZADO

EL LLANO AGUASCALIENTES A 22 DE SEPTIEMBRE DE 2026

TABLA DE CONTENIDO

TEMA 4.1 SUBTEMA 3 VISUALIZACIÓN DE LOS COMPONENTES.1

REFERENCIAS3

13

LISTA DE FIGURAS

Figura 1Infogravía Teorema de Bayes**Error! Bookmark not defined.**

ACTIVIDAD 2 (Práctica)

Tarea1. Revisión y replicación de ejemplos base

El estudiante deberá revisar, analizar y replicar los ejemplos presentados en la Unidad 1 correspondientes a: Aprendizaje supervisado, Aprendizaje no supervisado, Aprendizaje por refuerzo

Aprendizaje supervisado

Ejemplo código:

U1_supervisado_ejemplo.py U

aaa > U1_supervisado_ejemplo.py > ...

```python
# ============================================================
# Librerías necesarias
# ============================================================

import time                                 # Medir tiempo de ejecución
import warnings                             # Ignorar advertencias
import pandas as pd                         # Manejo de datasets (dataframes)
import seaborn as sns                       # Visualización estadística
import matplotlib.pyplot as plt             # Gráficos (matriz de confusión)

# Scikit-learn (Machine Learning)
from sklearn.model_selection import train_test_split   # División entrenamiento/prueba
from sklearn.neural_network import MLPClassifier       # Red Neuronal Perceptrón Multicapa
from sklearn.metrics import confusion_matrix           # Matriz de confusión

# Configuración: ignorar warnings futuros
warnings.filterwarnings("ignore", category=FutureWarning)

# ============================================================
# Encabezado del modelo
# ============================================================

print("\n" + "-" * 80)
print(" MODELO DE CLASIFICACIÓN CON UNA RED NEURONAL")
print("-" * 80 + "\n")
print(" DETECCIÓN DE VOCABULARIO DE CIBERSUICIDIO")

# ============================================================
# Carga del dataset
# ============================================================

inicio_tiempo = time.time()   # Inicia contador de tiempo

dataset = './Inputs/dataset_cibersuicidio.csv'
df = pd.read_csv(dataset)

print("\nDimensión del Dataset:")
print(f"Registros: {df.shape[0]}")
print(f"Columnas:  {df.shape[1]}\n")

print("Variables predictoras (X):")
X = df[df.columns[:-1]].values   # Todas las columnas excepto la última
print(X)

print("\nVariable a predecir (Y):")
y = df[df.columns[-1]].values    # Última columna
print(y)

# Mostrar características utilizadas
print("\nCaracterísticas utilizadas en el análisis:")
print(df.keys())

```

Aprendizaje supervizado

U1_supervisado_ejemplo.py U

aaa > U1_supervisado_ejemplo.py > ...

```
# ============================================================
# División de datos en entrenamiento y prueba
# ============================================================

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

print("\nEntrenamiento:", len(X_train), "registros (70%)")
print("Pruebas:", len(X_test), "registros (30%)")

# ============================================================
# Definición y entrenamiento del modelo
# ============================================================

# Red Neuronal con 1 capa oculta de 4 neuronas y hasta 11,000 iteraciones
red = MLPClassifier(max_iter=11000, hidden_layer_sizes=(4))
red.fit(X_train, y_train)

print(f"\nAprendizaje del algoritmo en el entrenamiento: {red.score(X_test, y_test):.2%}\n")

# Modelo adicional para comparación
model = MLPClassifier()
print("Ajusta el modelo de datos al proceso de entrenamiento")
model.fit(X_train, y_train)

# ============================================================
# Matriz de confusión
# ============================================================

y_pred = red.predict(X_test)
cnf_matrix = confusion_matrix(y_test, y_pred)

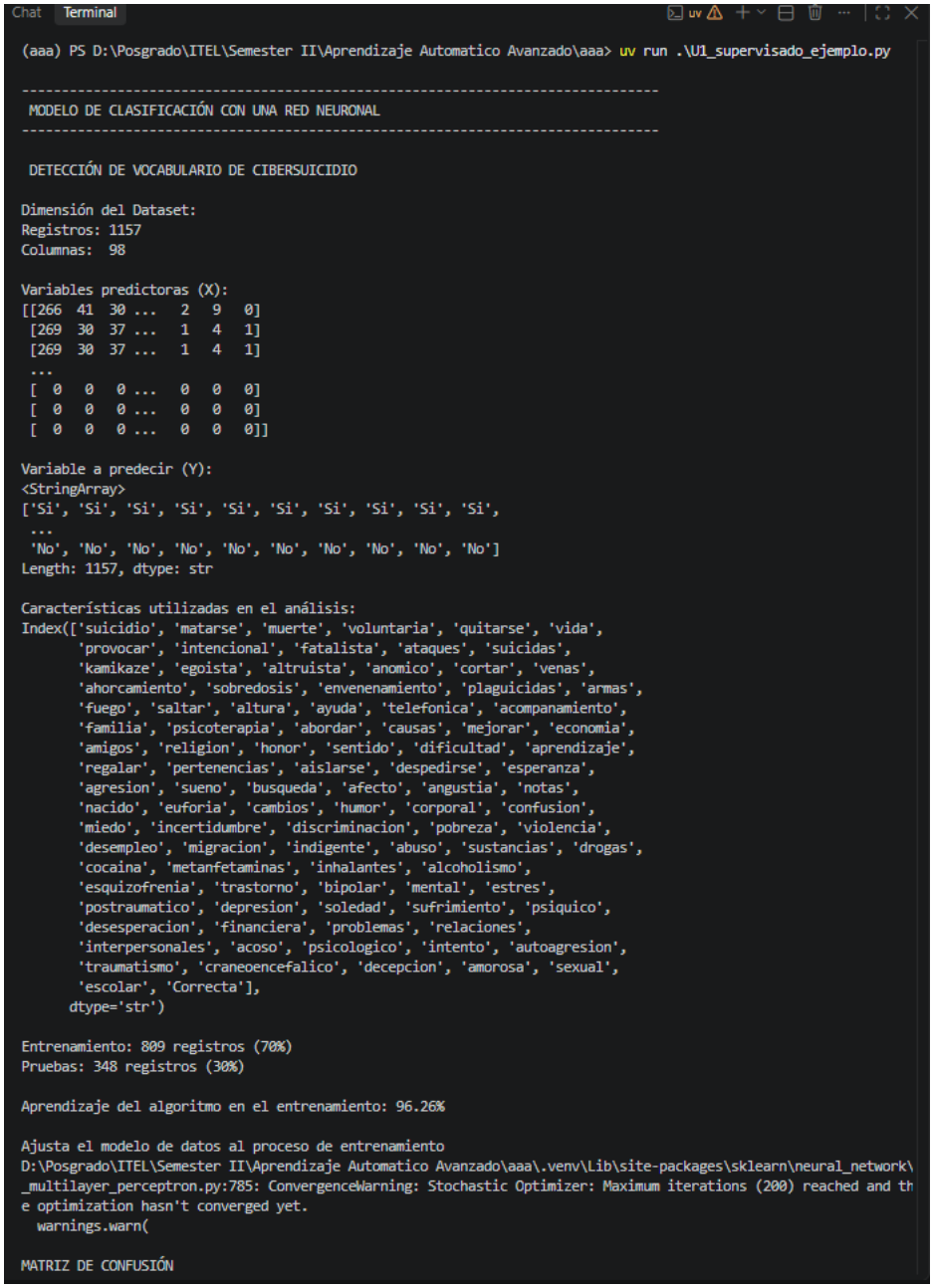
print("\nMATRIZ DE CONFUSIÓN")
sns.heatmap(cnf_matrix.T, square=True, annot=True, fmt='d', cbar=True)
plt.xlabel('Clase verdadera')
plt.ylabel('Clase predicha')
plt.title('Matriz de Confusión')
plt.show()

print("\nDescripción de la matriz:")
print("Verdaderos Negativos:", cnf_matrix[0, 0])
print("Verdaderos Positivos:", cnf_matrix[1, 1])
print("Falsos Positivos:", cnf_matrix[1, 0])
print("Falsos Negativos:", cnf_matrix[0, 1])
```

```
# ============================================================
# Resultados finales
# ============================================================

tiempo = time.time() - inicio_tiempo
precision = 1 - (y_test != y_pred).sum() / y_test.shape[0]

print("\nRESULTADOS DE CLASIFICACIÓN")
print("-" * 66)
print(f"Porcentaje de precisión: {precision * 100:.1f}%")
print(f"Errores de clasificación: {(y_test != y_pred).sum()} errores sobre {y_test.shape[0]} casos")
print(f"Tiempo de ejecución: {tiempo:.2f} segundos")
print("-" * 66)
```



```
Chat  Terminal

(aaa) PS D:\Posgrado\ITEL\Semester II\Aprendizaje Automatico Avanzado\aaa> uv run .\U1_supervisado_ejemplo.py

--------------------------------------------------------------------------------
 MODELO DE CLASIFICACIÓN CON UNA RED NEURONAL
--------------------------------------------------------------------------------

 DETECCIÓN DE VOCABULARIO DE CIBERSUICIDIO

Dimensión del Dataset:
Registros: 1157
Columnas:  98

Variables predictoras (X):
[[266  41  30 ...   2   9   0]
 [269  30  37 ...   1   4   1]
 [269  30  37 ...   1   4   1]
 ...
 [  0   0   0 ...   0   0   0]
 [  0   0   0 ...   0   0   0]
 [  0   0   0 ...   0   0   0]]

Variable a predecir (Y):
<StringArray>
['Si', 'Si', 'Si', 'Si', 'Si', 'Si', 'Si', 'Si', 'Si', 'Si',
 ...
 'No', 'No', 'No', 'No', 'No', 'No', 'No', 'No', 'No', 'No']
Length: 1157, dtype: str

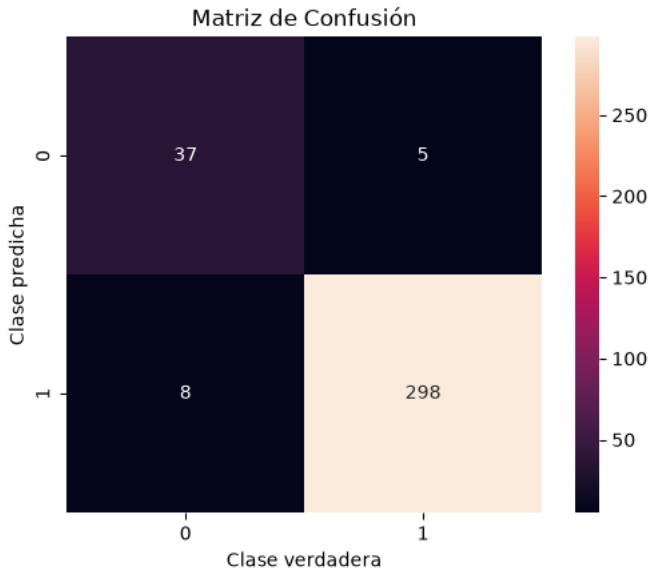
Características utilizadas en el análisis:
Index(['suicidio', 'matarse', 'muerte', 'voluntaria', 'quitarse', 'vida',
       'provocar', 'intencional', 'fatalista', 'ataques', 'suicidas',
       'kamikaze', 'egoista', 'altruista', 'anomico', 'cortar', 'venas',
       'ahorcamiento', 'sobredosis', 'envenenamiento', 'plaguicidas', 'armas',
       'fuego', 'saltar', 'altura', 'ayuda', 'telefonica', 'acompanamiento',
       'familia', 'psicoterapia', 'abordar', 'causas', 'mejorar', 'economia',
       'amigos', 'religion', 'honor', 'sentido', 'dificultad', 'aprendizaje',
       'regalar', 'pertenencias', 'aislarse', 'despedirse', 'esperanza',
       'agresion', 'sueno', 'busqueda', 'afecto', 'angustia', 'notas',
       'nacido', 'euforia', 'cambios', 'humor', 'corporal', 'confusion',
       'miedo', 'incertidumbre', 'discriminacion', 'pobreza', 'violencia',
       'desempleo', 'migracion', 'indigente', 'abuso', 'sustancias', 'drogas',
       'cocaina', 'metanfetaminas', 'inhalantes', 'alcoholismo',
       'esquizofrenia', 'trastorno', 'bipolar', 'mental', 'estres',
       'postraumatico', 'depresion', 'soledad', 'sufrimiento', 'psiquico',
       'desesperacion', 'financiera', 'problemas', 'relaciones',
       'interpersonales', 'acoso', 'psicologico', 'intento', 'autoagresion',
       'traumatismo', 'craneoencefalico', 'decepcion', 'amorosa', 'sexual',
       'escolar', 'Correcta'],
      dtype='str')

Entrenamiento: 809 registros (70%)
Pruebas: 348 registros (30%)

Aprendizaje del algoritmo en el entrenamiento: 96.26%

Ajusta el modelo de datos al proceso de entrenamiento
D:\Posgrado\ITEL\Semester II\Aprendizaje Automatico Avanzado\aaa\.venv\Lib\site-packages\sklearn\neural_network\
_multilayer_perceptron.py:785: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (200) reached and th
e optimization hasn't converged yet.
  warnings.warn(

MATRIZ DE CONFUSIÓN
```



Ejemplo código Aprendizaje no supervisado

U1_nosupervisado_ejemplo.py U

aaa > U1_nosupervisado_ejemplo.py > ...

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans

# ============================================================
# 1) Cargar dataset
# ============================================================
df = pd.read_csv("./Inputs/dataset_cibersuicidio.csv")

# ============================================================
# 2) Variables predictoras (sin etiqueta)
# ============================================================
X = df.drop(columns=["Correcta"], errors="ignore")

# ============================================================
# 3) Escalamiento
# ============================================================
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# ============================================================
# 4) PCA (2 componentes)
# ============================================================
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_scaled)

print("Varianza explicada por PC1 y PC2:", pca.explained_variance_ratio_)
print("Varianza acumulada (2 componentes):", pca.explained_variance_ratio_.sum())

# ============================================================
# 5) K-means clustering
# ============================================================
k = 3  # número de clusters
kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X_pca)

cluster_counts = pd.Series(clusters).value_counts().sort_index()
print("\nTamaño por cluster:")
print(cluster_counts)

```

```python

# ============================================================
# 6) Visualización PCA + KMeans
# ============================================================
plt.figure()
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters, s=15)
plt.title(f"PCA (2D) + K-means (k={k})")
plt.xlabel("Componente principal 1")
plt.ylabel("Componente principal 2")
plt.show()

# ============================================================
# 7) Interpretación de clusters
# ============================================================
X_with_cluster = X.copy()
X_with_cluster["cluster"] = clusters
cluster_means = X_with_cluster.groupby("cluster").mean(numeric_only=True)

top_n = 10
for c in range(k):
    top_features = cluster_means.loc[c].sort_values(ascending=False).head(top_n)
    print(f"\nTop {top_n} términos por frecuencia promedio en cluster {c}:")
    print(top_features)

# ============================================================
# 8) Comparación con etiqueta (opcional)
# ============================================================
if "Correcta" in df.columns:
    df_tmp = df.copy()
    df_tmp["cluster"] = clusters
    crosstab = pd.crosstab(df_tmp["cluster"], df_tmp["Correcta"], normalize="index")
    print("\nDistribución de la etiqueta 'Correcta' por cluster (solo interpretación):")
    print(crosstab)

```

```
(aaa) PS D:\Posgrado\ITEL\Semester II\Aprendizaje Automatico Avanzado\aaa> uv run .\U1_nosupervisado_ejemplo.py
Varianza explicada por PC1 y PC2: [0.1831769  0.06125512]
Varianza acumulada (2 componentes): 0.24443201348888166

Tamaño por cluster:
0      60
1    1091
2       6
Name: count, dtype: int64

Top 10 términos por frecuencia promedio en cluster 0:
suicidas     113.083333
suicidio      61.016667
vida          31.133333
sentido       20.716667
muerte        16.050000
familia       15.166667
mental        14.266667
problemas     11.016667
ayuda         10.666667
depresion     10.566667
Name: 0, dtype: float64

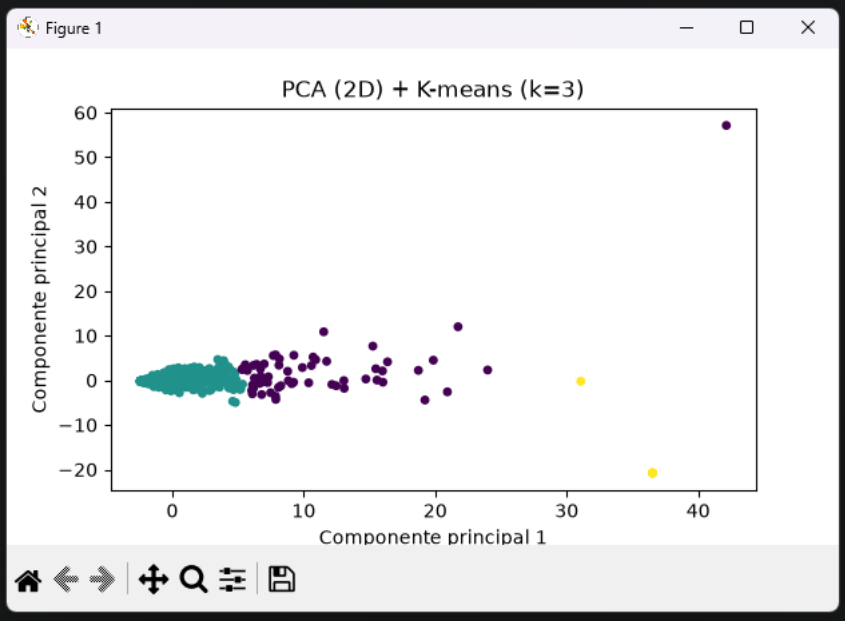
Top 10 términos por frecuencia promedio en cluster 1:
suicidas     13.247479
suicidio     10.210816
vida          5.060495
muerte        2.959670
ayuda         2.002750
mental        1.807516
sentido       1.674610
matarse       1.508708
familia       1.467461
problemas     1.327223
Name: 1, dtype: float64

Top 10 términos por frecuencia promedio en cluster 2:
suicidas      519.666667
suicidio      268.500000
vida           67.833333
ayuda          65.333333
mental         41.000000
muerte         35.833333
matarse        31.833333
causas         30.833333
trastorno      29.000000
relaciones     26.166667
Name: 2, dtype: float64

Distribución de la etiqueta 'Correcta' por cluster (solo interpretación):
Correcta        No        Si
cluster
0         0.000000  1.000000
1         0.150321  0.849679
2         0.000000  1.000000
```

```
(aaa) PS D:\Posgrado\ITEL\Semester II\Aprendizaje Automatico Avanzado\aaa> uv run .\U1_nosupervisado_ejemplo.py
Varianza explicada por PC1 y PC2: [0.1831769  0.06125512]
Varianza acumulada (2 componentes): 0.24443201348888166

Tamaño por cluster:
0      60
1    1091
2       6
Name: count, dtype: int64
```



Ejemplo código Aprendizaje por Refuerzo

U1_porRefuerzo_ejemplo.py U

aaa > U1_porRefuerzo_ejemplo.py > ...

```python
import numpy as np
import gymnasium as gym

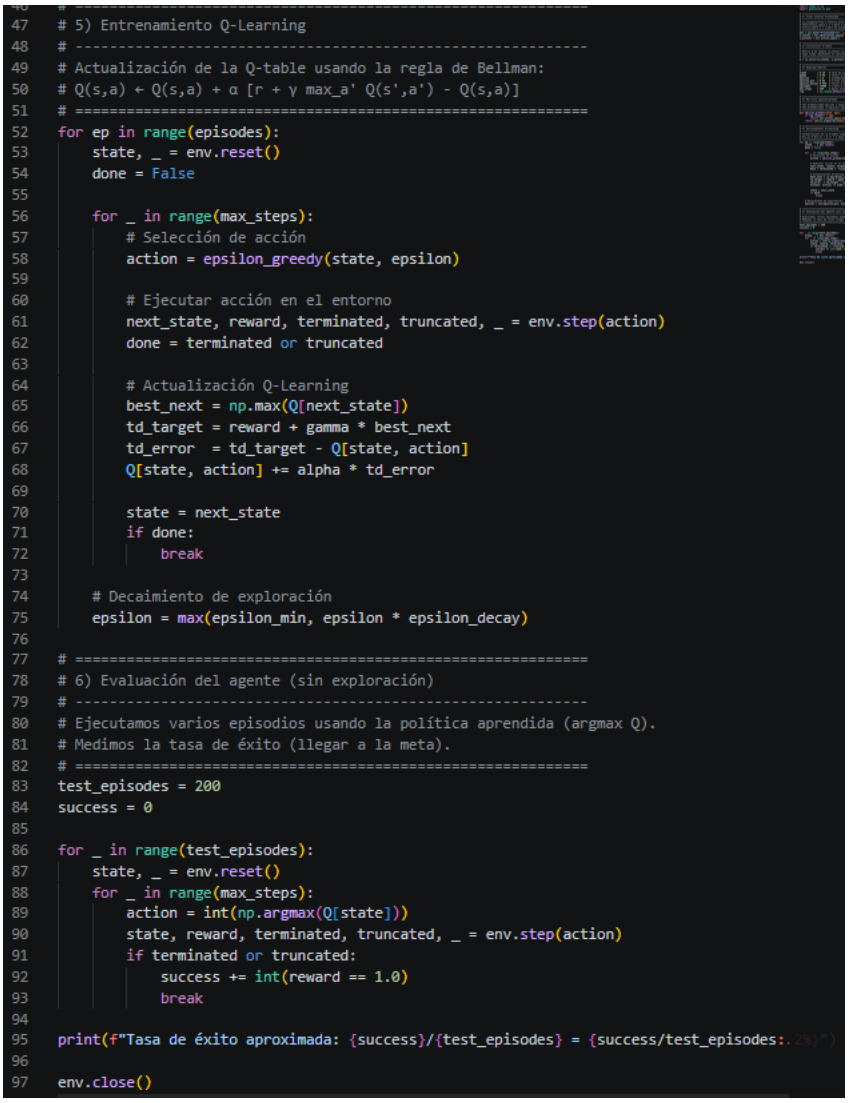
# ============================================================
# 1) Crear entorno FrozenLake
# ------------------------------------------------------------
# is_slippery=True → entorno estocástico (más realista, movimientos pueden fallar).
# observation_space.n → número de estados posibles (casillas del lago).
# action_space.n → número de acciones posibles (arriba, abajo, izquierda, derecha).
# ============================================================
env = gym.make("FrozenLake-v1", is_slippery=True)
n_states = env.observation_space.n
n_actions = env.action_space.n

# ============================================================
# 2) Inicializar Q-table
# ------------------------------------------------------------
# Matriz Q de tamaño [n_states x n_actions], inicializada en ceros.
# Cada celda representa el valor esperado de tomar una acción en un estado.
# ============================================================
Q = np.zeros((n_states, n_actions))

# ============================================================
# 3) Hiperparámetros
# ------------------------------------------------------------
alpha        = 0.10   # tasa de aprendizaje
gamma        = 0.99   # factor de descuento (importancia de recompensas futuras)
epsilon      = 1.0    # probabilidad inicial de exploración
epsilon_min  = 0.05   # mínimo valor de epsilon
epsilon_decay= 0.999  # factor de decaimiento de epsilon por episodio
episodes     = 15000  # número de episodios de entrenamiento
max_steps    = 200    # pasos máximos por episodio
rng          = np.random.default_rng(42)  # generador aleatorio reproducible

# ============================================================
# 4) Política epsilon-greedy
# ------------------------------------------------------------
# Con probabilidad epsilon → explorar (acción aleatoria).
# Con probabilidad (1-epsilon) → explotar (acción con mayor valor Q).
# ============================================================
def epsilon_greedy(state, eps):
    if rng.random() < eps:
        return env.action_space.sample()
    return int(np.argmax(Q[state]))

```



```python
# 5) Entrenamiento Q-Learning
# ------------------------------------------------------------
# Actualización de la Q-table usando la regla de Bellman:
# Q(s,a) ← Q(s,a) + α [r + γ max_a' Q(s',a') - Q(s,a)]
# ============================================================
for ep in range(episodes):
    state, _ = env.reset()
    done = False

    for _ in range(max_steps):
        # Selección de acción
        action = epsilon_greedy(state, epsilon)

        # Ejecutar acción en el entorno
        next_state, reward, terminated, truncated, _ = env.step(action)
        done = terminated or truncated

        # Actualización Q-Learning
        best_next = np.max(Q[next_state])
        td_target = reward + gamma * best_next
        td_error  = td_target - Q[state, action]
        Q[state, action] += alpha * td_error

        state = next_state
        if done:
            break

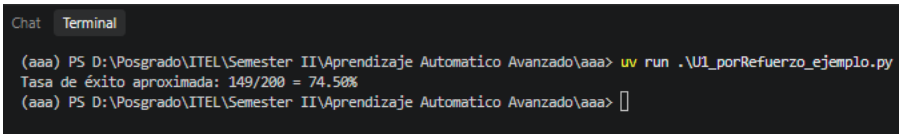
    # Decaimiento de exploración
    epsilon = max(epsilon_min, epsilon * epsilon_decay)

# ============================================================
# 6) Evaluación del agente (sin exploración)
# ------------------------------------------------------------
# Ejecutamos varios episodios usando la política aprendida (argmax Q).
# Medimos la tasa de éxito (llegar a la meta).
# ============================================================
test_episodes = 200
success = 0

for _ in range(test_episodes):
    state, _ = env.reset()
    for _ in range(max_steps):
        action = int(np.argmax(Q[state]))
        state, reward, terminated, truncated, _ = env.step(action)
        if terminated or truncated:
            success += int(reward == 1.0)
            break

print(f"Tasa de éxito aproximada: {success}/{test_episodes} = {success/test_episodes:.2%}")

env.close()
```


Chat
Terminal
(aaa) PS D:\Posgrado\ITEL\Semester II\Aprendizaje Automatico Avanzado\aaa> uv run .\U1_porRefuerzo_ejemplo.py
Tasa de éxito aproximada: 149/200 = 74.50%
(aaa) PS D:\Posgrado\ITEL\Semester II\Aprendizaje Automatico Avanzado\aaa>

REFERENCIAS

Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, *7*(12), 1–30. https://jmlr.org/papers/v7/demsar06a.html

ANEXOS

ANEXO 1. Código en Google Colab todas la Unidades.
https://drive.google.com/drive/folders/1qezpHlmrIUNsfi-sEQeEwjsYGa0lKdP2?usp=drive_link

ANEXO 2. Cóidog en Google Colab, unidad IV.
https://drive.google.com/drive/folders/1kq1RGlMqDb9ai8U5cZ6VRYmqCflRZnt_?usp=drive_link